

Automatisierter Sicherheitsbericht

Erstellt am: 17.05.2026 21:18:56

1. Management-Zusammenfassung

Im vorliegenden Scan wurden mehrere Seiten der Anwendung `http://juice-shop:3000` analysiert. Dabei wurden drei sicherheitsrelevante Schwachstellen festgestellt, von denen zwei als hochkritisch bis kritisch einzustufen sind:

1. SQL Injection im Login-Bereich mit erfolgreichem Login-Bypass
2. Reflected Cross-Site Scripting (XSS) in der Suchfunktion
3. Fehlender Schutz gegen Brute-Force-Angriffe auf das Login

Die kritischste Feststellung ist die SQL Injection im Login, da sie laut Rohdaten bereits zu einem erfolgreichen Zugriff mit Administratorrechten geführt hat. Das ist ein unmittelbares Risiko für die vollständige Kompromittierung der Anwendung und ihrer Daten.

Zusätzlich wurde in der Suchfunktion eine XSS-Schwachstelle nachgewiesen, bei der ein eingebetteter JavaScript-Payload ausgeführt wurde. Dadurch könnten Angreifer Benutzer im Browser angreifen, Sitzungen übernehmen oder Inhalte manipulieren.

Der fehlende Brute-Force-Schutz erhöht das Risiko automatisierter Passwortangriffe erheblich, insbesondere wenn bereits bekannte oder wiederverwendete Zugangsdaten existieren.

Insgesamt ist der Sicherheitszustand der untersuchten Anwendung aus Sicht dieses Scans kritisch, da mindestens eine Schwachstelle direkt zur Umgehung der Authentifizierung geführt hat.

2. Technischer Überblick über die analysierten Seiten

Im Scanlauf wurden folgende Zielseiten betrachtet:

- `http://juice-shop:3000/#`
- `http://juice-shop:3000/#/about`
- `http://juice-shop:3000/#/chatbot`
- `http://juice-shop:3000/#/complain`
- `http://juice-shop:3000/#/contact`
- `http://juice-shop:3000/#/login`
- `http://juice-shop:3000/#/photo-wall`

Erkannte Formulare und Interaktionspunkte

Zur fachlichen Einordnung wurden auch Formularfunde berücksichtigt:

- Login-Seite `/#/login`
- Felder: `email`, `password`, `remember me`
- Relevanz: Authentifizierung, direkt betroffen von SQL Injection und Brute-Force-Thematik
- Kontaktseite `/#/contact`
- Felder: Author, Comment, Range, CAPTCHA
- Im Scan keine direkt bestätigte Schwachstelle
- Beschwerdeseite `/#/complain`
- Felder: Customer, Message, File Upload
- Im Scan keine direkt bestätigte Schwachstelle
- Der Dateiupload ist grundsätzlich sicherheitsrelevant, aber aus den vorliegenden Rohdaten wurde keine bestätigte Upload-Schwachstelle abgeleitet
- Chatbot `/#/chatbot`
- Feld: `message`
- Im Scan keine bestätigte Schwachstelle

HTTP-Header-Kontext

Mehrere Antworten enthalten u. a.:

- `access-control-allow-origin: \*`
- `x-content-type-options: nosniff`
- `x-frame-options: SAMEORIGIN`

Diese Header liefern Kontext, sind hier aber nicht als eigenständige Schwachstellen zu bewerten, da aus den Rohdaten keine konkrete Ausnutzbarkeit abgeleitet werden kann. Insbesondere ist `Access-Control-Allow-Origin: \*` ohne weitere Informationen zu sensiblen CORS-Antworten allein noch keine belastbare Schwachstellenbestätigung.

3. Priorisierte Liste der gefundenen Schwachstellen

Priorität 1 - Kritisch

SQL Injection mit Login-Bypass

- Betroffene Seite: `http://juice-shop:3000/#/login`
- Risiko: Sehr hoch
- Begründung: Erfolgreiche Umgehung des Logins und Erhalt eines Tokens mit Admin-Bezug laut Rohdaten

Priorität 2 - Hoch

Cross-Site Scripting (XSS) in der Suchfunktion

- Betroffene Seite: `http://juice-shop:3000/#/search?q=...`
- Risiko: Hoch
- Begründung: JavaScript-Ausführung im Browser wurde bestätigt

Priorität 3 - Mittel

Fehlender Brute-Force-Schutz im Login

- Betroffene Seite: `http://juice-shop:3000/#/login`
- Risiko: Mittel
- Begründung: 40 fehlgeschlagene Login-Versuche ohne erkennbare Schutzreaktion

4. Detailanalyse pro Schwachstelle

4.1 SQL Injection mit Login-Bypass

- Betroffene Seite bzw. URL:
`http://juice-shop:3000/#/login`

In den Rohdaten ist der Schwachstellen-Eintrag zwar mit `http://juice-shop:3000/#/search` verknüpft, der Kontext aus Formular, Parameter `email` und Beschreibung zeigt jedoch klar, dass die Schwachstelle im Login-Prozess festgestellt wurde.

- Typ der Schwachstelle:
SQL Injection

- Technische Beobachtung:
Im Feld `email` wurde der Payload

`' OR 1=1 --`

verwendet. Die Rohdaten beschreiben ausdrücklich:

"Login bypass via SQL injection".

Zusätzlich wurde unter `credentials` ein JWT gespeichert, das auf einen erfolgreichen Login mit der E-Mail `admin@juice-sh.op` und Rolle `admin` hindeutet. Das ist ein starkes Indiz dafür, dass die Injektion nicht nur vermutet, sondern tatsächlich erfolgreich ausgenutzt wurde.

- Mögliche Auswirkungen:
- Umgehung der Authentifizierung
- Unbefugter Zugriff auf Benutzerkonten
- Möglicher Administratorzugriff
- Zugriff auf sensible Daten
- Manipulation oder Löschung von Daten
- Potenziell vollständige Kompromittierung der Anwendung

- Geschätzte CVSS 4.0-Bewertung:

9.8 / 10 (kritisch, geschätzt)

Begründung:

- Netzwerkbasiert ausnutzbar
- Geringe Komplexität
- Keine legitimen Rechte erforderlich
- Führt unmittelbar zu gravierendem Verlust von Vertraulichkeit, Integrität und potenziell auch Verfügbarkeit

Eine exakte CVSS-4.0-Vektorbildung ist mit den Rohdaten nur eingeschränkt möglich, insbesondere hinsichtlich nachgelagerter Systemauswirkungen. Die Bewertung wurde daher vorsichtig, aber aufgrund des erfolgreichen Login-Bypass sehr hoch angesetzt.

- Priorität:

Sofort / Kritisch

- Empfohlene Gegenmaßnahmen:

- Konsequente Verwendung parametrisierter SQL-Abfragen / Prepared Statements
- Keine direkte Einbettung von Benutzereingaben in Datenbankabfragen
- Serverseitige Eingabevalidierung
- Zentrale ORM-/Query-Builder-Prüfung auf unsichere Query-Erzeugung
- Detaillierte Code-Review des Authentifizierungs- und Benutzerabfragepfads
- Rotation gefährdeter Tokens/Sitzungen
- Überprüfung, ob unbefugte Anmeldungen bereits erfolgt sind
- Logging und Alarmierung für Authentifizierungsanomalien

- Leicht verständliche Erklärung:

Hier konnte das Login offenbar mit einem manipulierten Texteingabewert ausgetrickst werden. Statt korrekter Zugangsdaten wurde eine Zeichenfolge eingegeben, die die Datenbankabfrage verändert hat. Dadurch hat das System den Angreifer möglicherweise wie einen gültigen Benutzer behandelt - laut Rohdaten sogar mit Administratorrechten. Das ist so, als würde ein Türschloss nicht den echten Schlüssel prüfen, sondern sich durch einen Trick einfach öffnen lassen.

4.2 Cross-Site Scripting (XSS) in der Suchfunktion

- Betroffene Seite bzw. URL:

`http://juice-shop:3000/#/search?q=%3Ciframe%20src%3D%22javascript:alert(%60xss%60)%22%3E`

- Typ der Schwachstelle:

Reflected Cross-Site Scripting (XSS), nach aktuellem Datenstand wahrscheinlich reflektiert

- Technische Beobachtung:

Als Payload wurde verwendet:

`<iframe src='javascript:alert(`xss`)'>`

Der Scanner dokumentiert:

"Alert message is triggered at input".

Das ist ein klarer Hinweis darauf, dass Eingaben aus dem Suchparameter im Browser so verarbeitet wurden, dass JavaScript ausgeführt werden konnte.

- Mögliche Auswirkungen:

- Ausführung von JavaScript im Browser eines Opfers
- Session-Diebstahl, sofern Schutzmechanismen fehlen
- Phishing innerhalb der Anwendung
- Manipulation von Inhalten im Browser
- Umleitung auf schädliche Inhalte
- Missbrauch der Vertrauensstellung der legitimen Website

- Geschätzte CVSS 4.0-Bewertung:

6.8 / 10 (hoch bis oberes mittel, geschätzt)

Begründung:

XSS erfordert in der Regel, dass ein Benutzer einen manipulierten Link aufruft oder mit präparierten Inhalten interagiert. Die Schwachstelle ist klar bestätigt, aber die tatsächliche Auswirkung hängt von Browser-Schutzmechanismen, Session-Handling, CSP und weiteren Faktoren ab, die hier nicht vollständig bekannt sind. Deshalb keine noch höhere Bewertung.

- Priorität:

Hoch

- Empfohlene Gegenmaßnahmen:

- Konsequentes Output-Encoding abhängig vom Ausgabekontext
- Keine ungeprüfte Übernahme von URL-Parametern in das DOM
- Einsatz sicherer Framework-Mechanismen gegen DOM-/Reflected-XSS
- Verbot gefährlicher HTML-/JavaScript-Konstrukte durch Sanitization
- Einführung bzw. Härtung einer Content Security Policy (CSP)
- Sicherheitstests speziell für Such- und Filterparameter

- Leicht verständliche Erklärung:

In der Suchfunktion konnte statt normalem Suchtext ein schädlicher Code eingeschleust werden, der im Browser ausgeführt wurde. Für Benutzer bedeutet das: Ein Angreifer könnte einen manipulierten Link verschicken, der beim Öffnen unerwartete Aktionen auf der Website ausführt - etwa Inhalte verändern oder Benutzersitzungen angreifen.

4.3 Fehlender Brute-Force-Schutz im Login

- Betroffene Seite bzw. URL:

`http://juice-shop:3000/#!/login`

- Typ der Schwachstelle:

Fehlender Schutz gegen Brute-Force-/Credential-Stuffing-Angriffe

- Technische Beobachtung:

Laut Rohdaten wurden 40 fehlgeschlagene Login-Versuche durchgeführt. Dabei gab es:

- keine Änderung des Statuscodes
- keine erkennbare Änderung des Response-Bodys
- keinen `Retry-After`-Hinweis

Das spricht dafür, dass keine effektive Sperre, Verzögerung oder Drosselung aktiv war.

- Mögliche Auswirkungen:

- Automatisiertes Durchprobieren von Passwörtern
- Erhöhte Gefahr von Kontoübernahmen
- Missbrauch wiederverwendeter Passwörter
- Verstärkte Wirksamkeit von Credential-Stuffing-Angriffen

- Erhöhte Last auf dem Authentifizierungssystem

- Geschätzte CVSS 4.0-Bewertung:

5.9 / 10 (mittel, geschätzt)

Begründung:

Die Schwachstelle allein kompromittiert noch kein Konto, senkt aber die Hürde für Angriffe erheblich. Die tatsächliche Kritikalität hängt stark von Passwortqualität, MFA, IP-Limits, WAF-Regeln und Monitoring ab, zu denen keine belastbaren Daten vorliegen.

- Priorität:

Mittel, zeitnah beheben

- Empfohlene Gegenmaßnahmen:

- Rate Limiting für Login-Versuche

- Temporäre Kontosperre oder progressive Verzögerungen nach Fehlversuchen

- Schutz gegen Credential Stuffing auf IP-, Konto- und Geräteebene

- MFA für privilegierte Konten, idealerweise für alle Benutzer

- Monitoring und Alarmierung bei auffälligen Login-Serien

- CAPTCHA oder zusätzliche Prüfungen nach verdächtigen Fehlversuchsmustern

- Leicht verständliche Erklärung:

Die Login-Seite scheint nicht ausreichend zu bremsen, wenn sehr oft hintereinander falsche Passwörter ausprobiert werden. Dadurch können Angreifer automatisiert viele Kombinationen testen, bis sie eventuell zufällig oder mit gestohlenen Daten Erfolg haben.

5. Konkrete Handlungsempfehlungen

Sofortmaßnahmen

1. SQL Injection im Login unverzüglich beheben

- Authentifizierungslogik prüfen

- Alle Datenbankzugriffe im Login auf parametrisierte Queries umstellen

- Notfall-Code-Review des Login-Pfads durchführen

2. Sicherheitsvorfall prüfen

- Da ein Token mit Admin-Bezug vorliegt, sollte von einer möglichen Kompromittierung ausgegangen werden

- Logs auf unbefugte Anmeldungen, Rollenänderungen und Datenzugriffe prüfen
- Ggf. Sessions/Tokens invalidieren

3. Privilegierte Konten absichern

- Admin-Zugänge zurücksetzen
- MFA aktivieren, sofern möglich
- Passwortrotation für kritische Konten

Kurzfristige Maßnahmen

4. XSS in der Suche beheben

- Ausgabe von Suchparametern nur kontextgerecht kodiert darstellen
- DOM-basierte Einbindung von Benutzereingaben absichern
- CSP einführen oder verschärfen

5. Brute-Force-Schutz einführen

- Rate Limits
- Delays nach Fehlversuchen
- Erkennung und Blockierung auffälliger Muster
- Schutz besonders für Admin- und privilegierte Konten

Mittelfristige Maßnahmen

6. Sicherheitsprüfungen auf weitere Eingabepunkte ausweiten

- Kontaktformular
- Beschwerdeformular inklusive Dateupload
- Chatbot-Eingaben

7. Secure Development Practices stärken

- Standardisierte serverseitige Validierung
- Sichere ORM-/Datenbanknutzung
- Automatisierte Sicherheits-Tests in CI/CD
- Regelmäßige Penetrationstests

8. Header- und Browser-Schutzmaßnahmen härten

- CSP ergänzen
- Prüfung weiterer moderner Sicherheitsheader
- CORS-Konfiguration auf notwendige Ursprünge beschränken, falls sensible API-Antworten betroffen sind

6. Gesamtfazit

Der Scan zeigt ein ernstzunehmendes Sicherheitsrisiko für die Anwendung. Besonders schwerwiegend ist die SQL Injection im Login, da sie nach den vorliegenden Rohdaten direkt zu einem Admin-Login-Bypass geführt hat. Diese Schwachstelle muss als kritischer Befund mit unmittelbarem Handlungsbedarf betrachtet werden.

Die zusätzlich bestätigte XSS-Schwachstelle in der Suche eröffnet weitere Angriffsmöglichkeiten gegen Endbenutzer und kann in Kombination mit anderen Schwächen erheblichen Schaden verursachen.

Der fehlende Brute-Force-Schutz verschärft die Lage, da der Login-Bereich dadurch auch für automatisierte Angriffe unnötig offensteht.

Zusammengefasst lautet die Bewertung:

- Gesamtrisiko: Hoch bis kritisch
- Dringlichkeit: Sofortige Behebung der Login-bezogenen Schwachstellen
- Empfohlene Reihenfolge:
 1. SQL Injection beheben und Vorfallanalyse durchführen
 2. XSS beseitigen
 3. Brute-Force-Schutz nachrüsten
 4. Weitere Eingabepunkte gezielt nach ähnlichen Fehlern prüfen

Wenn gewünscht, kann ich im nächsten Schritt aus diesen Ergebnissen auch noch einen formaleren Audit-Bericht im Stil eines Kundenreports oder eine tabellarische Executive-Summary mit Risikomatrix erstellen.